

interface is configured with a special NIC configuration that will have no IP for that interface, and will allow all packets via that specially configured NIC. Moreover, the VM packets reaching the Internet via this specially configured NIC will have the OpenStack tenant router's gateway IP address as the source in each packet.

As I mentioned earlier, in the limitations above, AWS will only allow the packet flow when the MAC address is known/registered in its environment. Also, the IP address must match the MAC address.

In our case, the packet from the above-mentioned OpenStack tenant router will have the IP address of the router's gateway in every single packet, and the packet source's MAC address will be the MAC address of the router's interface.

 **Note:** These details are available if you use *ip netns* show followed by *ip netns exec qrouter-<router_ID> ifconfig* commands in the OpenStack controller's terminal. Since the MAC address is unknown or not registered in the AWS environment, the packets will be dropped when they reach the AWS switch. To allow the VM packets to reach the Internet via the AWS switch, we need to use some tricks/hacks in our OpenStack set-up.

Making use of what we have

One possible way is to register the router's MAC address and its IP address with the AWS environment. However, this is not feasible. As of now, AWS does not have the feature of registering any random MAC address and IP address inside the VPC. Moreover, allowing this type of functionality will be a severe security threat to the environment.

The other method is to make use of what we have. Since we have used a special type of interface configuration for the provider NIC, you will note that the IP address assigned to the provider NIC (say, *eth1*) is left unused. We could use this available/unused IP address for the OpenStack router's gateway. The command given below will do the trick:

```
neutron router-gateway-set router provider --fixed-ip ip_
address=<Registered_IP_address>
```

IP address and MAC address mismatches

After configuring the router gateway with the AWS registered IP address, each packet from the router's gateway will have the AWS registered IP address as the source IP address, but with the unregistered MAC address generated by the OVS. As I mentioned earlier, while discussing the limitations of AWS, the IP address must match the MAC address registered; else, all the packets with the mismatched MAC and IP address will be dropped by the AWS switch. To make the registered MAC address match with the IP address, we need to change the MAC address of the router's interface.

The following steps will do the magic.

Step 1) Install the *macchanger* tool.

Step 2) Note down the actual/original MAC address of the provider NIC (*eth1*).

Step 3) Change the MAC address of the provider NIC (*eth1*).

Step 4) Change the MAC address of the router's gateway interface to the original MAC address of *eth1*.

Step 5) Now, try to ping 8.8.8.8 from the router namespace.

If you get a successful ping response, then we are done with the Cloud-on-the-Cloud set-up.

Key points to remember

Here are some important things that one needs to keep track of.

Changing the MAC address: In my case, I had used the Ubuntu 14.04 LTS server, with which there was no issue in changing the MAC address using the *macchanger* tool. However, when I tried the Ubuntu 16.04 LTS, I got an error saying, "No permission to modify the MAC address." I suspect the error was due to the *cloud-init* tool not allowing the MAC address' configuration. So, before setting up OpenStack, try changing the MAC address of the NIC.

Floating IP disabled: Associating a floating IP to any of OpenStack's VMs will send the packet via the router's gateway with the source IP address as the floating IP's IP address. This will make the packets hit the AWS switch with a non-registered IP and MAC address, which results in the packets being dropped. So, I could not use the floating IP functionality in this set-up. However, I could access the VM publicly using the following NAT process.

Using NAT to access OpenStack's VM: As mentioned earlier, I could access the OpenStack VM publicly using the registered IP address that was assigned to the router's gateway. Use the following NAT command to access the OpenStack VM using the AWS EC2 instance's elastic IP:

```
$ ip netns exec qrouter-f85bxxxx-61b2-xxxx-xxxx-ba0xxxx
iptables -t nat -A PREROUTING -p tcp -d 172.16.20.101
--dport 522 -j DNAT --to-destination 192.168.20.5:22
```

 **Note:** In the above command, I had NAT forwarding for all packets for 172.16.20.101 with Port 522. Using the above NAT command, all the packets reaching 172.16.20.101 with Port number 522 are forwarded to 192.168.20.5:22.

Here, 172.16.20.101 is the registered IP address of the AWS EC2 instance which was assigned to the router's gateway. 192.168.20.5 is the local IP of the OpenStack VM. Notably, 172.16.20.101 already has NAT with the AWS elastic IP, which means all the traffic that comes to the elastic IP (public IP) will be forwarded to this VPC